

Visión general del curso

Bienvenido al curso de **Aprendizaje profundo con TensorFlow y Keras**. Estamos entusiasmados de que te unas a nosotros en este viaje para dominar las técnicas de Aprendizaje profundo utilizando algunas de las herramientas más poderosas de la actualidad.

Requisitos previos

Para sacar el máximo provecho de este curso, usted debe sentirse cómodo con los siguientes temas y tecnologías:

- Este curso requiere conocimientos básicos de programación en Python.
- Familiaridad con conceptos matemáticos básicos como gradientes y matrices.

Si necesitas aprender más sobre este tema antes de tomar este curso, el siguiente curso te ofrecerá la experiencia que necesitas para tener éxito en éste:

- [Aprendizaje automático con Python](#)

Objetivos del curso

Después de completar este curso, serás capaz de:

- Realizar operaciones tensoriales en TensorFlow.
- Implementar y entrenar modelos de regresión lineal utilizando Keras y TensorFlow.
- Explicar los conceptos de Regresión logística y aplicarlos a problemas de clasificación utilizando Keras.
- Manejar datos y entrenar modelos y optimizarlos usando gradient descent en Keras.

Esquema del curso

Este curso consta de siete módulos, que se enumeran a continuación. Te animamos a que dediques varias horas cada semana para completar con éxito todos los módulos en 12 semanas. La constancia le ayudará a alcanzar sus objetivos de aprendizaje

Usted obtendrá el máximo beneficio de ver todos los videos y lecturas y solidificar ese conocimiento completando todas las actividades:

- **Módulo 1: Funcionalidades Avanzadas de Keras**

Este módulo proporciona una visión general de las funcionalidades avanzadas de Keras. Cubrirá la API funcional de Keras para la creación de modelos complejos y la creación de capas y modelos personalizados en Keras. A continuación, el módulo describe la integración de Keras con TensorFlow 2.x para una funcionalidad mejorada.

- **Módulo 2: CNN avanzadas en Keras**

En este módulo, aprenderá a desarrollar redes neuronales convolucionales (CNNs) avanzadas utilizando Keras. Aprenderá técnicas de Aumento de datos con Keras. Además, implementará el Aprendizaje por transferencia con Keras y aprovechará los modelos preentrenados. Por último, aprenderá a utilizar TensorFlow para mejorar las capacidades de procesamiento de imágenes.

- **Módulo 3: Transformadores en Keras**

Este módulo cubre la construcción y entrenamiento de Transformers avanzados usando Keras. Desarrollará modelos transformadores para datos secuenciales y series temporales utilizando TensorFlow con Keras. Además, aprenderá a implementar técnicas avanzadas de Transformer para la generación de texto.

- **Módulo 4: Aprendizaje no supervisado y Modelos generativos en Keras**

En este módulo, aprenderá los principios del Aprendizaje no supervisado en Keras. Aprenderá a construir y entrenar autoencoders y modelos de difusión. Además, desarrollará redes generativas adversarias (GAN) utilizando Keras e integrará TensorFlow para tareas avanzadas de Aprendizaje no supervisado.

- **Módulo 5: Técnicas Avanzadas de Keras y Proyecto del Curso**

En este módulo, aprenderá técnicas avanzadas para el desarrollo de modelos en Keras. Creará bucles de entrenamiento personalizados, optimizará modelos utilizando Keras y realizará ajustes de hiperparámetros con Keras Tuner. Por último, aprenderá a utilizar TensorFlow para la optimización de modelos y bucles de entrenamiento personalizados.

- **Módulo 6: Introducción al Aprendizaje de refuerzo con Keras**

En este módulo aprenderá los fundamentos del Aprendizaje de refuerzo y sus aplicaciones en Keras. El módulo también cubre los algoritmos de Q-Learning utilizando Keras. Desarrollará y entrenará redes Q profundas (DQNs) con Keras para tareas avanzadas de aprendizaje de refuerzo.

- **Módulo 7: Proyecto Final y Evaluación**

En este módulo, implementará el proyecto final e intentará realizar la evaluación final.

Herramientas/Software

En este curso, utilizarás versiones gratuitas o pruebas de varias herramientas, incluyendo:

- Python y Notebook de Jupyter
- Librerías Keras y TensorFlow

El curso no requiere que pagues para completar ninguna actividad. Es posible que desee actualizar a una versión de pago, aunque deberá incurrir en esos costes.

Consejos para el éxito

- Mantente organizado y controla los plazos.
- Practica regularmente la codificación y experimenta con los ejemplos proporcionados.
- Participa en la comunidad y haz preguntas si necesitas ayuda.

Enhorabuena por haber dado estos pasos para ampliar tus conocimientos y tu carrera profesional Disfruta de tu viaje

Marcar como completo

Me gusta

No me gusta

Informar de un problema

#####

INTRODUCCIÓN KERAS AVANZADO

DEEP LEARNING MODULO 1

#####

• DEEP LEARNING ES UN RECONOCIMIENTO
MUCHOS CAMPOS:

- Robótica
- Visión por computador
- Procesamiento del lenguaje natural.

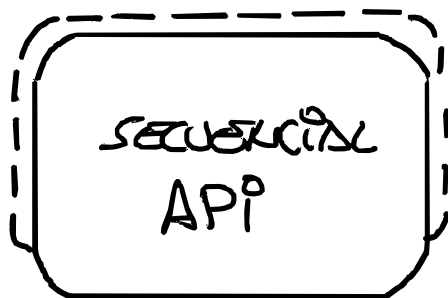
• **OBJETIVOS:** DPI DE REDES NEURONALES DE
ALTO NIVEL ESCRITO EN PYTHON.

• WHAT YOU WILL LEARN?

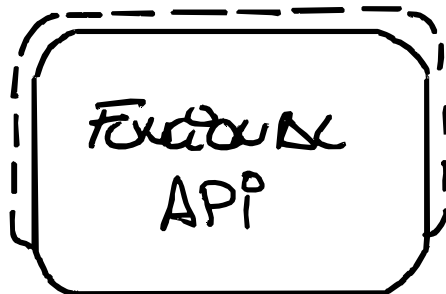
- FUNCIONES PERSONALIZADAS KERAS
- EXTENDER LA FUNCIONALIDAD DE LA DPI KERAS
EN LA CREACIÓN DE MODELOS COMPLEJOS
- ILUSTRAR LOS APLICACIONES REALES

- KERAS SE USA AMPLIAMENTE EN LA INDUSTRIA Y EN EL MUNDO ACADEMICO PARA MUCHAS APLICACIONES.

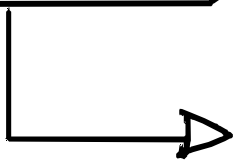
* KERAS DIFERENTES FEATURES *



- EXCELENTE PARA PCA LINEAL SIMPLES DE CAPAS



- PROPORCIONA MÁS FLEXIBILIDAD Y CONTROL
- PARA MODELOS COMPLEJOS



- PUEDE CREAR MODELOS CON VARIAS ENTRADAS Y SALIDAS
- CAPAS COMBINADAS
- FLUJOS DE DATOS NO SECUENCIALES

* EJEMPLO API SEQUENTIAL *

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
# Create a Sequential model
```

```
model = Sequential([  
    Dense(64, activation='relu', input_shape=(784,)),  
    Dense(10, activation='softmax')  
])
```

```
# Compile the model
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])
```

1. LA PRIMERA CAPA DEPENDE DE AGREGAR COMO LA ACTIVACION DE:
- RELU
2. LA SEGUNDA CAPA ES LA DE SALIDA Y SE ACTIVA CON:
- SOFTMAX (CLASIFICACION).

* EJEMPLO FUNCIONA! API Keras *

```
from tensorflow.keras.models import Model  
from tensorflow.keras.layers import Input, Dense
```

Define the input

```
inputs = Input(shape=(784,))
```

Define layers

```
x = Dense(64, activation='relu')(inputs)
```

```
outputs = Dense(10, activation='softmax')
```

Create a model

```
model = Model(inputs=inputs, outputs=outputs)
```

Compile the model

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])
```

1. SE DEFINEN LAS CAPAS DE ENTRADA
2. LAS CAPAS OCULTAS
3. LAS CAPAS DE SALIDA.

* VENTAJAS DE UNA API FUNCTION *

- MUY IMPORTANTES PARA LA INVESTIGACIÓN Y EL DESARROLLO DE MODELOS DE VANGUARDIA
- ES FLEXIBLE:
 - PERMITE CREAR ARQUITECTURAS COMPLEJAS
 - └─▶ . MODELOS DE SUPERSERVISORES MÚLTIPLES
 - . CONEXIONES RESPONSIVAS
 - . ETC...
- PROPORCIONA CLARIDAD
- LA ESTRUCTURA DEL MODELO ES MÁS EXPLICITA Y FÁCIL DE DEPURAR.
- LAS CAPAS Y LOS MODELOS SE PUEDEN REEMPLAZAR EN DIFERENTES CAPAS DE LA ARQUITECTURA.

* MODEL COMPLETES MULTIPLE INPUT *

from tensorflow.keras.layers import concatenate

Define the input

inputA = Input(shape=(64,))

inputB = Input(shape=(128,))

First branch operates on the first input

x = Dense(8, activation='relu')(inputA)

x = Dense(4, activation='relu')(x)

The second branch operates on the second input

y = Dense(16, activation='relu')(inputB)

y = Dense(4, activation='relu')(y)

Combine the outputs of the two branches

combine_c = concatenate([x, y])

Apply a dense layer and then a regression
prediction on the combined outputs

$z = \text{Dense}(z, \text{activation} = \text{'relu'})$ (combined)

$z = \text{Dense}(1, \text{activation} = \text{'linear'})$ (z)

The model will accept the inputs of the two branches and outputs a single value

$\text{model} = \text{Model}(\text{inputs} = [\text{inputA}, \text{inputB}], \text{outputs} = z)$

Compile the model

$\text{model.compile}(\text{optimizer} = \text{'adam'}, \text{loss} = \text{'mean-square-error'})$

* APLICACIONES EL MUNDO REAL *

- MEDICINA: DETECTAR ENFERMEDADES
- CON ANÁLISIS DE IMÁGENES

- FINANZAS: PREDECIR TENDENCIAS DE MERCADO

- CONDUCCIÓN AUTÓNOMA:
- TAREAS DE PERCEPCIÓN

#####

API FUNCIONAL KERAS

API DE SUBCLASES.

MODULO 1

#####

* API FUNCIONAL KERAS *

- PERMITE CONSTRUIR MODELOS DE REDES NEURONALES COMPUESTOS DE FORMA MUY FLEXIBLE
- MODELOS CON MÚLTIPLES ENTRADAS Y SALIDAS
- CAPAS COMPARTIDAS
- MODELOS CON FLUJOS DE DATOS NO LINEALES

* SWT *
* SWT *

```
from tensorflow.keras.models import Model  
from tensorflow.keras.layers import Input, Dense
```

```
# Define the input
```

```
inputs = Input(shape=(784,))
```

```
# Define layers
```

```
x = Dense(64, activation='relu')(inputs)
```

```
outputs = Dense(10, activation='softmax')
```

```
# Create a model
```

```
model = Model(inputs=inputs, outputs=outputs)
```

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
```

Define the Input

```
Inputs = Input(shape=(784,))
```

Define layers

```
x = Dense(64, activation='relu')(Inputs)
outputs = Dense(10, activation='softmax')
```

Create a model

```
model = Model(inputs=Inputs, outputs=outputs)
```

Define the Input

```
Inputs = Input(shape=(784,))
```

• CAPA ENTRADA
• ESPECIFICA FORMAS

Define layers

```
x = Dense(64, activation='relu')(Inputs)
outputs = Dense(10, activation='softmax')
```

→ CAPA OCULTA

- N° NEURONAS
- ACTIVACIÓN

→ CAPA SALIDA

→ FUNCIÓN DE ACTIVACIÓN
DE CLASIFICACIÓN
MULTICLASES

* MODEL COMPLETES MULTIPLE INPUT *

from tensorflow.keras.layers import concatenate

Define the input

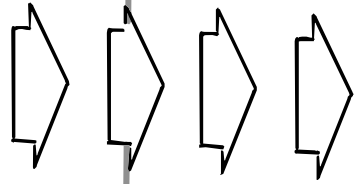
inputA = Input(shape=(64,))

inputB = Input(shape=(128,))

First branch operates on the first input

x = Dense(8, activation='relu')(inputA)

x = Dense(4, activation='relu')(x)



The second branch operates on the second input

y = Dense(16, activation='relu')(inputB)

y = Dense(4, activation='relu')(y)

Combine the outputs of the two branches

combine_c = concatenate([x, y])

Apply a dense layer and then a regression prediction on the combined outputs

z = Dense(2, activation='relu')(combine_c)

z = Dense(1, activation='linear')(z)

The model will accept the inputs of the two branches and outputs a single value

model = Model(inputs=[inputA, inputB], outputs=z)

Compile the model

model.compile(optimizer='adam', loss='mean-square-error')

• MULTIPLE ENTRIES Y SAÍDAS

• APRESENTA AS MULTIDRES

First branch operates on the first input

X = Dense (8, activation='relu')(input A)

X = Dense (4, activation='relu')(X)

X = Model(inputs = input A, outputs = X)

The second branch operates on the second input

Y = Dense (16, activation='relu')(input B)

Y = Dense (4, activation='relu')(Y)

Y = Model(inputs = input B, outputs = Y)

- SE CREAM LAS CAPAS OCULTAS Y SE FUNDEEN POR UN ÚNICO RESULTADO

The model will accept the inputs of the two branches and outputs a single value

model = Model(inputs = [input A, input B], outputs = Z)

- SE DEFINEN RAMAS SEPARADAS DEL MODELO
- CADA UNA PROCESANDO UNA DE LAS ENTRADAS

⦿ CAPAS COMPARTIDAS ⦿

```
from tensorflow.keras.layers import Lambda
```

```
# Define the input layer
```

```
input = Input(shape=(28, 28, 1))
```

```
# Define a shared convolutional base
```

```
conv_base = Dense(64, activation='relu')
```

```
# Process the input through the shared layer
```

```
processed_1 = conv_base(input)
```

```
processed_2 = conv_base(input)
```

```
# Create model using the shared layer
```

```
model = Model(inputs=input, outputs=[processed_1,  
                                     processed_2])
```

* EJEMPLO PRÁCTICO:

IMPLEMENTAR UN MODELO COMPLETO.

- MÚLTIPLES ENTRADAS
- CAPAS COMPARTIDAS
- SALIDA COMPARTIDAS

```
from tensorflow.keras.layers import Conv2D, Flatten,
from tensorflow.keras.activations import Maxpooling2D, relu, linear
```

First input model

```
inputA = Input(shape=(32, 32, 1))
```

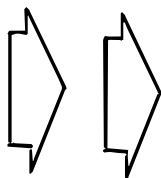
```
x = Conv2D(32, (3, 3), activation=relu)(inputA)
```

```
x = Maxpooling2D((2, 2))(x)
```

```
x = Flatten()(x)
```

```
x = Model(inputs=inputA, outputs=x)
```

Second input model



input B = Input(shape = (32, 32, 1))

y = Conv2D(32, (3, 3), activation = relu)(input B)

y = Maxpooling2D((2, 2))(y)

y = Flatten()(y)

y = Model(inputs = input B, outputs = y)

Combine the output of the two branches

combine = concatenate([x.output, y.output])

Apply a FC layer and then a regression prediction on the combined outputs

z = Dense(64, activation = relu)(combine)

z = Dense(1, activation = linear)(z)

The model will accept the inputs of the two branches and the output a single value

model = Model(inputs = [x.input, y.input], outputs = z)

* APP KERAS SUBCLASES *

- OFRECE MÁS FLEXIBILIDAD
- MODELOS PERSONALIZADOS Y DINÁMICOS
- SUBCLASIFICANDO LA CLASE DEL MODELO E IMPLEMENTANDO SU PROPIO MÉTODO:

└─> call()

- PARA CONSTRUIR MODELOS EN LOS QUE NOS SÓ PASES DESPUÉS EXISTENCIALMENTE EL PASO HACIA DELANTE
 - SE USA PARA BUCLES DE ENTRENAMIENTO PERSONALIZADO
 - PARA ARQUITECTURAS NO ESTÁNDAR.
-
-
-
-

SINTAXIS SUBCLASSES TERN

```
import tensorflow as tf
```

```
# Define your model by subclassing
```

```
class MyModel(tf.keras.Model):
```

```
    def __init__(self):
```

```
        super(MyModel, self).__init__()
```

```
    # Define layers
```

```
        self.dense1 = tf.keras.layers.Dense(64, act='relu')
```

```
        self.dense2 = tf.keras.layers.Dense(10, act='softmax')
```

```
    def call(self, inputs):
```

```
        # Forward pass
```

```
        x = self.dense1(inputs)
```

```
        return self.dense2(x)
```

```
# Instantiate the model
```

```
model = MyModel()
```

```
# Define loss function and optimizer
```

```
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy
```

```
optimizer = tf.keras.optimizers.Adam()
```

* CASOS DE USO SUBCLASES *

- Modelos con arquitecturas dinámicas

↳ Cuando las arquitecturas necesitan ser cambiadas dinámicamente

↳ En algunos modelos de aprendizaje de refuerzo

- Loops de entrenamiento

↳ Necesitas más control sobre el proceso de entrenamiento

↳ bucles personalizados

- Investigaciones y prototipos

- Experimentar con nuevas tipos de capas o arquitecturas que aún no están disponibles en la API standard de Keras

#####

LABORATÓRIO I

MODULO 1

#####

IMPLEMENTAÇÃO DE LA DPI
FUNÇÕES DE KERAS

#####

CREACIÓN DE CAPAS PERSONALIZADAS KERAS

MODULO 1

#####

- LAS CAPAS PERSONALIZADAS PERMITEN DEFIINIR TUS PROPIAS OPERACIONES EN UNA RED NEURAL.
- DÁNDOLES FLEXIBILIDAD DE ADAPTAR TUS MODELOS A TAREAS ESPECÍFICAS Y EXPERIMENTAR CON NUEVAS IDEAS.
- ENTENDER COMO FUNCIONAN LAS CAPAS CUSTOM PUEDE MEJORAR ENORMEMENTE TU CAPACIDAD PARA INICIAR PRESENTAR PROBLEMAS COMPLEJOS.
- CREAR CAPAS PERSONALIZADAS ES ESENCIAL CUANDO SE NECESITAN FUNCIONALIDADES ESPECÍFICAS. QUE KERAS NO PROPORCIONA.

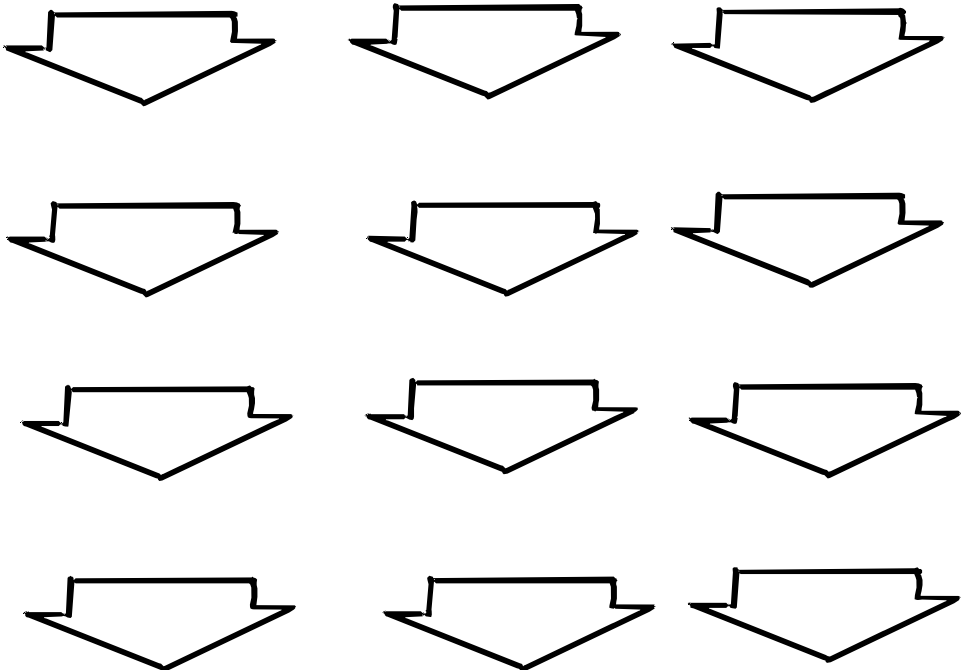
* LAS CAPAS ESTÁNDAR:

- DENSE
- CONVOLUTIONS
- LSTM

CUBREN UNA AMPLIA
GAMA DE CASOS DE
USO.

PERO A VECES SE NECESITA ALGO MÁS ESPECÍFICO

~~Y~~ NO USAR CAPAS PERSONALIZADAS USAMOS
TODO EL POTENCIAL DE KERAS, PERMITIENDO
MODELOS MÁS SOPHISTICADOS Y REFINADOS.



* BASICS CUSTOM LAYERS *

```
from tensorflow.keras.layers import Layer
```

```
class MyCustomLayer(Layer):
```

```
    def __init__(self, units = 32 ** Keras):
```

```
        super(MyCustomLayer, self).__init__(**Keras)
```

```
        self.units = units
```

```
    def build(self, input_shape):
```

```
        self.w = self.add_weight(shape=(input_shape  
                                         [-1], self.units),
```

```
                                trainable = True)
```

```
                                initializer = 'randn',
```

```
                                self.b = self.add_weight(shape=(self.units, ),
```

```
                                initializer = 'zeros',
```

```
                                trainable = True)
```

```
    def call(self, inputs):
```

```
        return tf.matmul(inputs, self.w) + self.b
```

```
import tensorflow as tf
```

```
print(tf.executing_eagerly()) # Return True in TF 2.X
```

```
# Simple tensor operation
```

```
a = tf.constant([1, 2, 3])
```

```
b = tf.constant([4, 5, 6])
```

```
result = tf.add(a, b)
```

```
print(result)
```

→ # outputs: [5, 7, 9]

* CUSTOM DENSE LAYER *

```
class CustomDenseLayer(Layer):
```

```
    def __init__(self, units = 32):
```

```
        super(CustomDenseLayer, self).__init__()
```

```
        self.units = units
```

```
    def build(self, input_shape):
```

```
        self.w = self.add_weight(shape = (input_shape
```

```
            [-1], self.units),
```

```
            initializer = 'random-normal',
```

```
            trainable = True)
```

```
        self.b = self.add_weight(shape = (self.units,),
```

```
            initializer = 'zeros',
```

```
            trainable = True)
```

```
    def call(self, inputs):
```

```
        return tf.nn.relu(tf.matmul(inputs, self.w) +
```

```
            self.b)
```

```
from tensorflow.keras.models import Sequential
```

```
model = Sequential([
```

```
    CustomDenseLayer(64),
```

```
    CustomDenseLayer(10)])
```

```
model.compile(optimizer = 'adam', loss = 'categorical_crossentropy')
```


#####

TENSORFLOW 2.X

VISION GENERAL

MODULO 1

#####

* TENSORFLOW ES UNA PLATAFORMA DE
CÓDIGO ABIERTO, DESARROLLADA POR
GOOGLE PARA MACHINE LEARNING Y
DEEP LEARNING

* SE PUEDE USAR EN VARIOS ENTORNOS:
- SERVIDORES
- DISPOSITIVOS PERIFÉRICOS

● FUNCIONES:

* EAGER EXECUTION: HACE QUE TENSORFLOW SEA
MÁS INSTINTIVO AL EJECUTAR
OPERACIONES DE FORMA INMEDIATA

* **APIs intuitivas de alto nivel:** integra
APIs como la API de alto nivel,
simplificando el proceso de creación y
entrenamiento de modelos.

* **Soporte para varias plataformas
y dispositivos:**

- dispositivos móviles
- WEB
- dispositivos integrados

* **Escalabilidad y optimización**

- Escalar sin problemas entre CPU, GPU, TPU
- Optimizar el rendimiento de las tareas
de machine learning a gran escala.

* **ecosistema rico:**

- amplia gama de herramientas y bibliotecas
que mejoran sus capacidades:
- tensorflow light
- tensorflow.js
- tensorflow extended tfx.

#####*#####

EAGER EXECUTION

#####*#####

(EJECUCIÓN RÁPIDA)

- PERMITE QUE LAS OPERACIONES SE EJECUTEN INMEDIATAMENTE SIN CREAR GRÁFICOS.
- SIMPLIFICA EL PROCESO DE DESPLAZACIÓN
- FACILITA LA PROGRAMACIÓN INTERACTIVA
- MUY ÚTIL PARA PRINCIPANTES Y COLABORADORES DÍKHMICOS.

* BENEFICIOS *

* DESPLAZACIÓN MEJORADA.

↳ FEEDBACK INMEDIATO

* CÓDIGO SIMPLIFICADO

↳ FÁCIL DE ENTENDER

```
import tensorflow as tf
print(tf.executing_eagerly())

# Simple tensor operation
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])
result = tf.add(a, b)
print(result) # Outputs: [5 7 9]
```

Returns True in TensorFlow 2.x

* PROGRAMACIÓN Y DESARROLLO INTERACTIVO

* APPS ALTO NÚMERO *

• INTEGRAL KERAS

→ SENCILLO ENTENDIMIENTO LA
CREACIÓN Y ENTRENAMIENTO DE
MODELOS DE DEEP LEARNING

→ INTERFAZ FÁCIL DE USAR

* BENEFICIOS INTEGRACIÓN DE KERAS *

- CREACIÓN DE MODELOS USANDO CÓDIGO SENCILLO Y CONCISO.
- COMPATIBLE CON MODELOS DE FORMA MODULAR:
↳ CAPAS Y BLOQUES SE PUEDEN COMBINAR
- AMPLIA DOCUMENTACIÓN

* EJEMPLO INTEGRACIÓN KERAS *

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# define a simple feed forward neural network
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)),
    Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

* ECOSISTEMA TENSORFLOW *

● TENSORFLOW LITE :

- PARA INTEGRAR MODELOS DE DEEP LEARNING EN DISPOSITIVOS MÓVILES E INTEGRADOS
- INFERENCIA DE BAJA RENDIMIENTO
- BAJA LATENCIA
- PERMITE EL APRENDIZAJE EN EL PROPIO DISPOSITIVO.

● TENSORFLOW.JS :

- PARA ENTRENAR E IMPLEMENTAR ENTORNOS DE JAVA SCRIPT Y SIN JS.

└─> - NAVEGADORES WEB

- PERMITE IMPLEMENTAR DEEP LEARNING EN LAS APLICACIONES WEB

● TENSORFLOW EXTENDED (TFX) :

- PLATAFORMA INTEGRAL PARA IMPLEMENTAR CAMPAÑAS DE DL DE PRODUCCIÓN
- DESARROLLO
- MONITOREO
- ADMINISTRACIÓN

● TENSORFLOW Hub:

- REPOSITORIO DE MÓDULOS DE MÁQUINAS LEARNING REUTILIZABLES
- SE PUEDEN INTEGRAR FÁCILMENTE EN LAS APLICACIONES DE TENSORFLOW PARA DESEMPEÑO DE DESARROLLO

● TENSOR BOARD:

- HERRAMIENTAS DE VISUALIZACIÓN PARA TENSORFLOW
- PROPORCIONA INFORMACIÓN SOBRE EL PROCESO DE ENTRENAMIENTO DEL MODELO

└─> ● GRÁFICOS
● MÉTRICAS
● OTROS DATOS ...

#####

LABORATORIO II MODULO 1

#####

CREACIÓN DE CAPAS Y MODELOS
PERSONALIZADOS.

- Keras es una API de redes neuronales de alto nivel escrita en Python y capaz de ejecutarse sobre TensorFlow, Theano y CNTK.
- Keras se utiliza ampliamente en la industria y el mundo académico para diversas aplicaciones, desde el reconocimiento de imágenes y del habla hasta los sistemas de recomendación y el Procesamiento de lenguaje natural.
- Keras Functional API ofrece ventajas como flexibilidad, claridad y reutilización.
- Puede utilizar la API funcional de Keras para desarrollar modelos en campos tan diversos como la sanidad, las finanzas y la conducción autónoma.
- La API Funcional de Keras permite definir capas y conectarlas en un gráfico de capas.
- La API Funcional puede manejar modelos con múltiples entradas y salidas.
- Otra potente característica de la API Funcional son las capas compartidas, que resultan útiles cuando se desea aplicar la misma transformación a múltiples entradas.
- La creación de capas personalizadas en Keras le permite adaptar sus modelos a necesidades específicas, implementar nuevas ideas de investigación y optimizar el rendimiento para tareas únicas.
- Practicando y experimentando con capas personalizadas, comprenderás mejor cómo funcionan las redes neuronales y mejorarás tu capacidad de innovación.
- TensorFlow 2.x es una plataforma potente y flexible para el aprendizaje automático con características como la ejecución ansiosa, API de alto nivel y un rico ecosistema de herramientas.
- La comprensión de estas características y capacidades le ayudará a construir y desplegar modelos de aprendizaje automático de manera más eficaz, ya sea trabajando en la investigación, la creación de prototipos, o aplicaciones de producción.

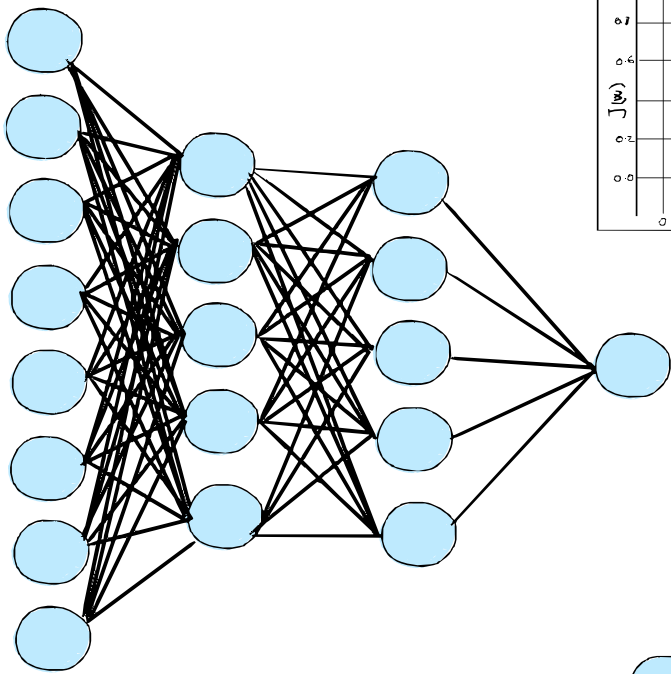
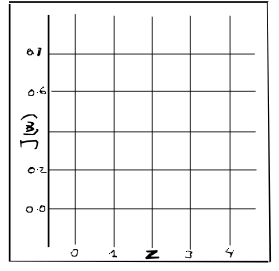
Glossary: Advanced Keras Functional API

Welcome! This alphabetized glossary contains many of the terms you'll find within this course. This comprehensive glossary also includes additional industry-recognized terms not used in course videos. These terms are important for you to recognize when working in the industry, participating in user groups, and participating in other certificate programs.

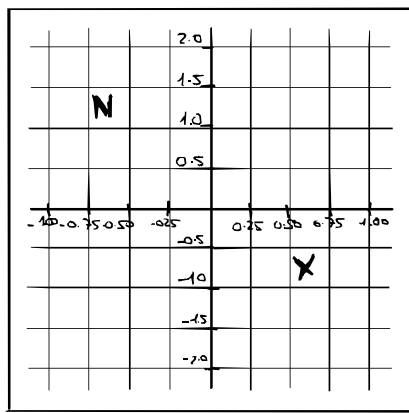
Term	Definition
Build	A method that creates the layer's weights, called once during the first invocation of the layer.
Call	A method that defines the forward pass logic of the layer.
Custom layer	A user-defined layer that allows customization of operations in a neural network, providing flexibility for specific tasks and experimentation.
Eager execution	A TensorFlow feature that executes operations immediately, making it more intuitive and useful for debugging and interactive programming.
Init	A method that initializes the layer's attributes.
Input layer	The first layer in a model that defines the input shape.
Keras	A high-level neural network API written in Python that can run on top of TensorFlow, Theano, and CNTK.
Keras Functional API	A powerful API for creating complex models with multiple inputs and outputs, shared layers, and non-sequential data flows.
Keras Sequential API	Creates models with layers in a linear stack.
ReLU	An activation function that outputs the input directly if positive; otherwise, it outputs zero. Commonly used in hidden layers.
Shared layer	Helpful when applying the same transformation to multiple inputs.
Softmax	An activation function suitable for classification tasks.
TensorFlow 2.x	An open-source platform for machine learning developed by Google, providing comprehensive tools to build and deploy machine learning models across various environments, from servers to edge devices.
TensorBoard	A visualization toolkit for TensorFlow that provides insights into the model training process, including metrics, graphs, and other useful data.
TensorFlow Extended (TFX)	An end-to-end platform for deploying production ML pipelines. TFX provides tools for model deployment, monitoring, and management, ensuring that machine learning models perform reliably in production environments.
TensorFlow Hub	A repository of reusable machine learning modules, which can be easily integrated into TensorFlow applications to accelerate development.
TensorFlow.js	A library for training and deploying machine learning models in JavaScript environments, such as web browsers and Node.js.
TensorFlow Lite	A lightweight framework for deploying machine learning models on mobile and embedded devices.



Skills Network



X



#####

